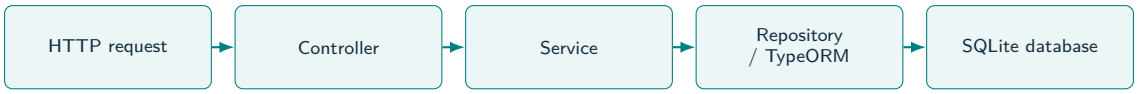
TP1 Defense Guide

What I built, why I built it, and how to explain it

Completed through slide 5 / TypeORM connection

This document describes the actual state of the project at handoff. It is designed for a technical explanation: every file has a role, every decorator has a reason, and the next unfinished exercise is clearly identified.



Remember The implementation currently demonstrates Nest modules, a custom value provider, constructor injection, one test endpoint, a TypeORM entity, and a working SQLite connection. Todo validation and CRUD are still future work.

Project	`todo-api`
Location	`/home/yassine/Desktop/TP1-NestJS/todo-api`
Framework	NestJS 11 with TypeScript
Database	SQLite through the supported `better-sqlite3` driver
Current endpoint	`GET /uuid-test` returns a newly generated UUID

1 The professor's expected reasoning

The TP1 instructions emphasize four habits. They are course requirements, not code features:

- Understand every line well enough to explain it without reading it for the first time during the interview.
- Think about architecture before adding code.
- Keep code coherent, readable, reusable, and properly divided into modules.
- Treat validation like a technical interview: explain decisions, tradeoffs, and failure cases.

When explaining this project, start with responsibility rather than syntax:

Piece	Responsibility
Module	Organizes controllers and providers and contro
Controller	Receives HTTP requests and returns response
Service	Contains application behavior.
Provider	Gives Nest a dependency under a token.
Entity	Maps a class to a database table.
TypeORM	Connects the entity to a database.

2 Project creation

The project was generated with the Nest CLI using a version compatible with the installed Node.js version. The generated application starts at `src/main.ts` and bootstraps `AppModule`.

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

`NestFactory.create(AppModule)` builds Nest's dependency graph. `listen(...)` starts the HTTP server. The root module is the assembly point; requests do not execute through the module class as an extra layer. The original generated route is in `src/app.controller.ts`. Its controller calls `AppService.getHello()`, which returns `Hello World!`. This proved that the application starts and routes requests correctly.

3 CommonModule and UUID provider

Why a common module?

The assignment asks for a module that will be used frequently. The project therefore has `src/common/common.module.ts`. It is marked global because it exposes a reusable dependency across the application.

```
@Global()
@Module({
  providers: [
    {
      provide: UUID_PROVIDER,
      useValue: uuidv4,
    },
  ],
  exports: [UUID_PROVIDER],
})
export class CommonModule {}
```

Why useValue?

The UUID library exports an existing function. Nest does not need to instantiate a class or calculate a one-time value. `useValue: uuidv4` places the function itself in the dependency-injection container. Each consumer can call it and obtain a new UUID. The project deliberately avoids `useValue: uuidv4()`. That would execute the generator once during startup and inject one fixed string. It also does not use `useClass`, because `uuidv4` is a function, not a class. The provider token is a symbol in `src/common/constants/common.constants.ts`:

```ts
export const UUID_PROVIDER = Symbol('UUID_PROVIDER');
```

The symbol is the runtime key. Since it is not a class, consumers must use `@Inject(UUID_PROVIDER)` explicitly.

4 Dependency-injection proof

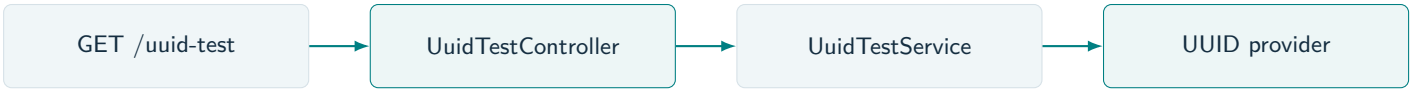
The test feature is under `src/uuid-test/`. It has a module, service, and controller. The service receives the UUID function through its constructor:

```ts
@Injectable()
export class UuidTestService {
  constructor(
    @Inject(UUID_PROVIDER)
    private readonly generateUuid: () => string,
  ) {}

  generate() {
    return { id: this.generateUuid() };
  }
}
```

The controller exposes the service through `GET /uuid-test`:

```ts
@Controller('uuid-test')
export class UuidTestController {
  constructor(private readonly uuidTestService: UuidTestService) {}

  @Get()
  generate() {
    return this.uuidTestService.generate();
  }
}
```

Refreshing the endpoint produces a different UUID because the injected value is a generator function and the service calls it for every request.



5 TodoEntity

The first database model is in `src/todos/entities/todo.entity.ts`. Its enum is in `src/todos/enums/status.enum.ts`. The assignment says the status enum was already defined in an earlier exercise; this project uses the temporary values `TODO`, `IN_PROGRESS`, and `DONE` until the course's exact enum is confirmed.

```ts
@Entity('todos')
export class TodoEntity {
  @PrimaryGeneratedColumn()
  id!: number;

  @Column()
  name!: string;

  @Column()
  description!: string;

  @CreateDateColumn()
  createdAt!: Date;

  @Column({ type: 'varchar', default: StatusEnum.TODO })
  status!: StatusEnum;
}
```

Decorator	Meaning
`@Entity('todos')`	Maps the class to the `todos` table.
`@PrimaryGeneratedColumn()`	Integer primary key generated by the database.
`@Column()`	Ordinary persisted column.
`@CreateDateColumn()`	Creation time managed by TypeORM.

An entity describes stored data. A later DTO will describe data allowed from an HTTP client. Generated fields such as `id` and `createdAt` must not be accepted as ordinary client input.

6 TypeORM configuration

The TypeORM setup is in `src/app.module.ts`:

```
TypeOrmModule.forRoot({
  type: 'better-sqlite3',
  database: 'todo.sqlite',
  entities: [TodoEntity],
  synchronize: true,
}),
```

`forRoot(...)` creates the application-level connection and tells TypeORM which entities to map. The local database is the file `todo.sqlite`. `synchronize: true` is suitable for this learning project, but it must not be used in production because automatic schema changes can lose data. The initial attempt used `type: 'sqlite'`, but the installed TypeORM version's type definitions support `better-sqlite3` instead. The driver was installed and the configuration was changed accordingly. The database file is now created successfully when the application starts.

Remember The future feature module will add `TypeOrmModule.forFeature([TodoEntity])`. That registers the Todo repository in the feature module so the service can inject it.

7 Verification and current limitations

Successful checks:

- `npm run build` passes.
- `todo.sqlite` is created.
- `GET /uuid-test` returns a UUID.
- Repeating the request returns a different UUID.

The generated unit tests for `UuidTestService` and `UuidTestController` currently fail because their isolated testing modules do not register the UUID token and service dependencies. The runtime application is working; these tests need mock providers or imported modules before the test suite can pass.

8 What remains in TP1

The next unfinished point is slide 6. The remaining work should be implemented in this order:

1. Add `CreateTodoDto`, validation decorators, centralized messages, and the global `ValidationPipe`.
2. Add an optional-field update DTO and validate the status enum.
3. Add `updatedAt` and `deletedAt`; keep `createdAt` outside update input.
4. Extract creation, update, and deletion timestamp columns into a reusable base entity.
5. Create `TodosModule`, `TodosService`, and `TodosController`; register the Todo repository with `forFeature`.
6. Implement create, update, hard delete, soft delete, restore, status counts, get-all, get-by-id, filtering, and pagination.

9 Short answers for an oral explanation

Question	Answer
Why is UUID a value provider?	The library gives us an existing function; Nest should inject that function.
Why is `@Inject` needed?	The token is a symbol, so Nest cannot infer it from a class type.
Why export the token?	A provider must be exported to be available outside its declaring module.
Why global CommonModule?	It is intended for frequent shared use, so importing it everywhere is unnecessary.
What does the entity do?	It maps the Todo class and its decorators to the database table structure.
What does `forRoot` do?	It configures the application's TypeORM connection.
What does `synchronize` do?	It synchronizes the development schema from entity metadata.
Why is the DTO separate?	Input rules and persisted database structure are different responsibilities.

Useful commands

```
npm run build
npm run format
npm run start:dev
curl http://localhost:3000/uuid-test
```